



**DIKTISAINTEK
BERDAMPAK**

COMPUTER VISION (TFP505)

Pertemuan 3 – Image Preprocessing Techniques

ALIFIA REVAN PRANANDA

Department of Information Technology
Faculty of Engineering
Universitas Tidar

IMAGE PREPROCESSING TECHNIQUES



Have you ever struggled with poor quality images in your machine learning or computer vision projects?

IMAGE PREPROCESSING TECHNIQUES

IMAGE PREPROCESSING is the process of manipulating raw image into a usable and meaningful format.

- ❑ It allows us to **eliminate unwanted distortions** and **enhance specific qualities** essential for computer vision applications.
- ❑ Preprocessing is a **crucial first step** to prepare your image data before feeding it into machine learning models.

Resize & Crop

We also can add padding before processing image

Normalization

Pixel values are scaled to a specific range for simplify the computation

Color Conversion

color conversion for better color-based analysis

Noise Removal

Removing unwanted object

Histogram Equalization

Balances the distribution of pixel intensity values.

Data Augmentation

Prevents overfitting and improves model generalization

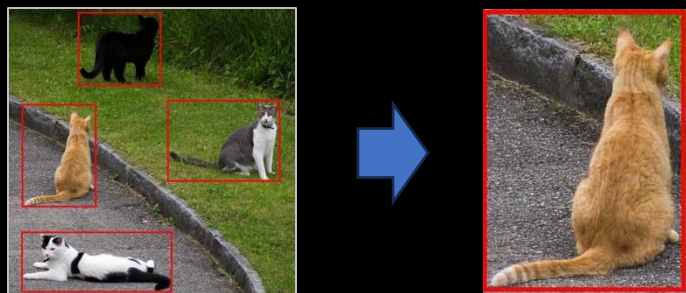
RESIZING & CROPPING

1 Resizing

Adjusts all images to a fixed size required by the model (e.g., 224×224 pixels for CNNs such as ResNet, VGG).

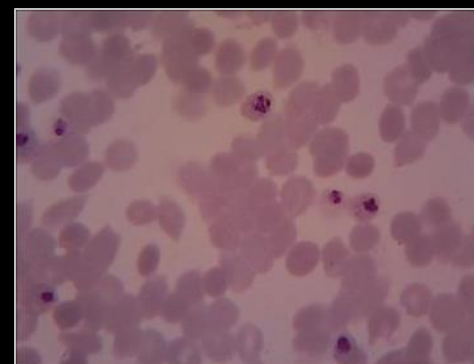
2 Cropping

Removes irrelevant background and focuses on the region of interest (ROI). Useful in object detection and face recognition.



3 Padding

Padding is the process of adding extra pixels (usually zeros, but sometimes other values) around the border of an image.



Original image with not standard format

Padding should be added so that the image become square as required for CNN models

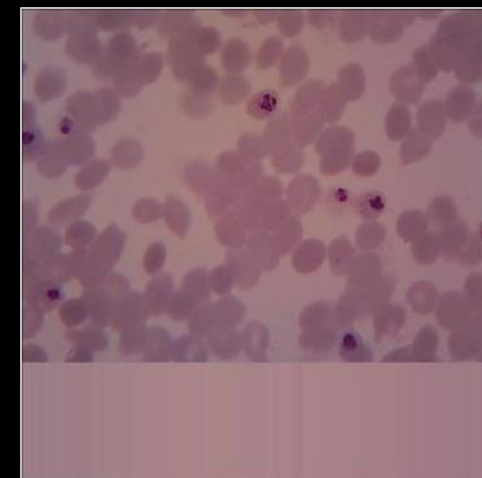
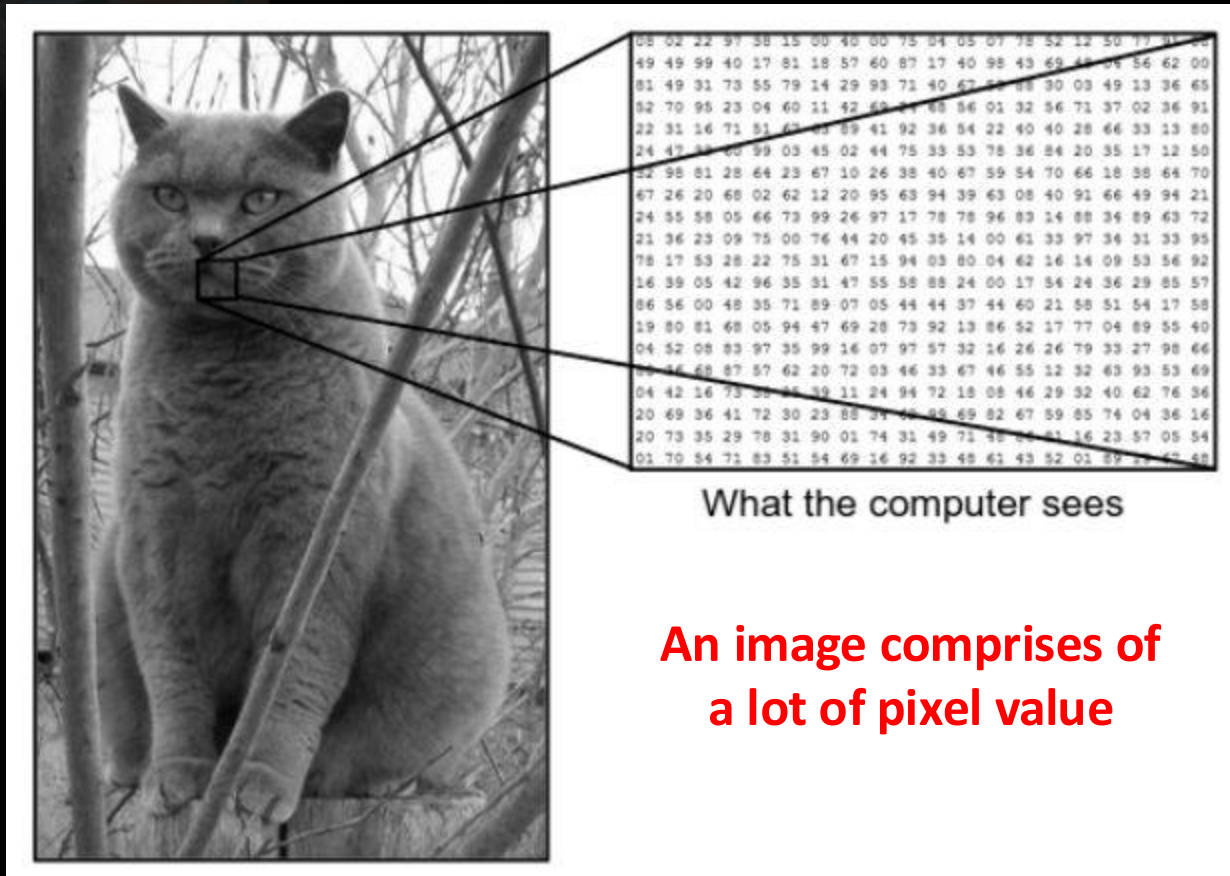


IMAGE NORMALIZATION



Processing an original image is often **challenging and computationally expensive** because it typically consists of three channels (RGB). Each pixel is represented by three separate values corresponding to red, green, and blue intensity, which **significantly increases the amount of data** to be handled.

- ❑ Image normalization is required **to reduce the computational burden by scaling pixel values** into a smaller and more consistent range.
- ❑ This step **simplifies the processing** of multi-channel data, since the three channels (RGB) can be standardized and treated more efficiently.

COLOR CONVERSION



Color conversion is another essential step in image pre-processing, especially when dealing with multi-channel RGB data.

By converting images into different color spaces, such as **Grayscale**, **HSV**, or **Lab**, we can reduce complexity or highlight specific visual information relevant to the task.

NOISE REMOVAL

Noise removal is a crucial pre-processing step to enhance image quality before further analysis. It aims to eliminate unwanted variations in pixel values caused by factors such as artefact, noise, low lighting, sensor limitations, or transmission errors.

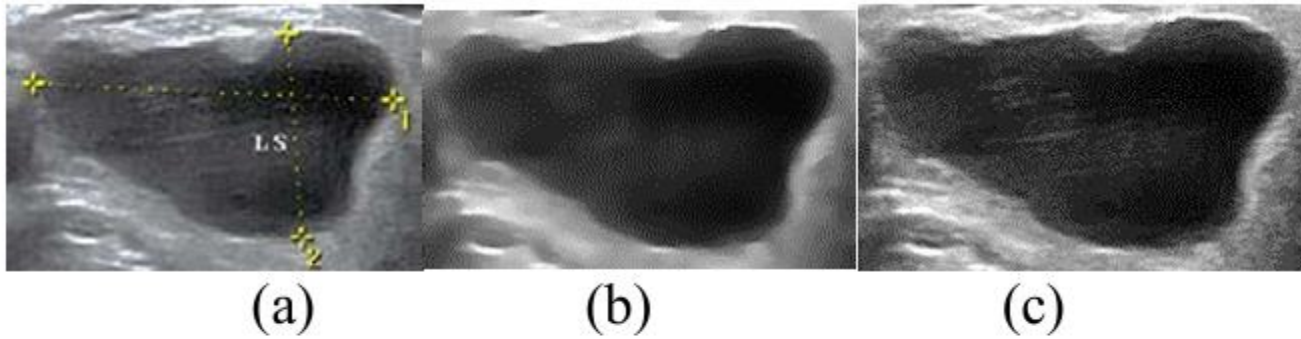


Figure 2. Result of preprocessing step (a) ROI image (b) adaptive median filter (c) SRBF.

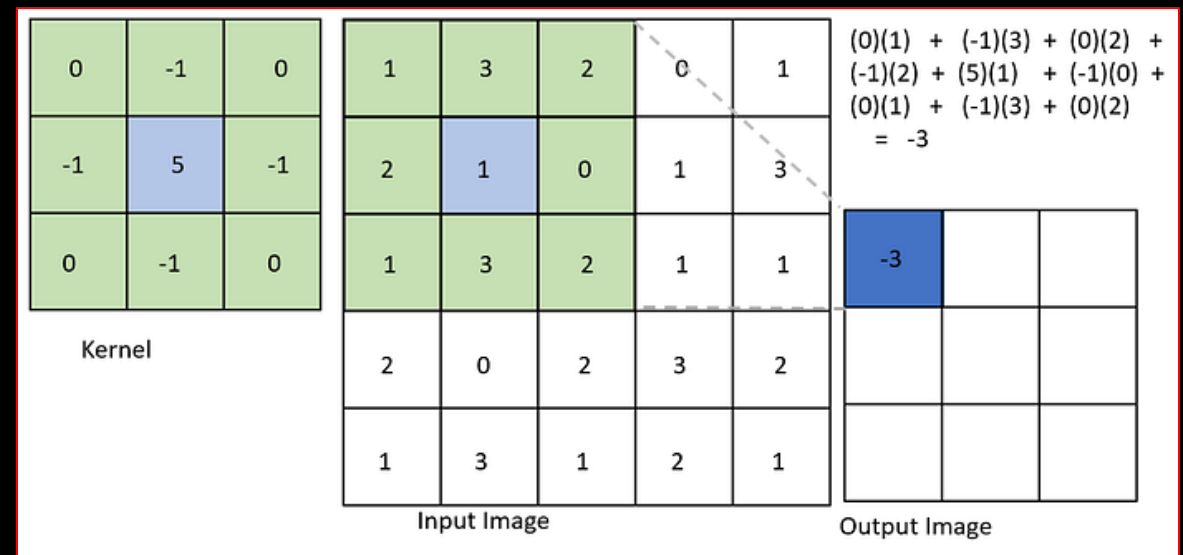
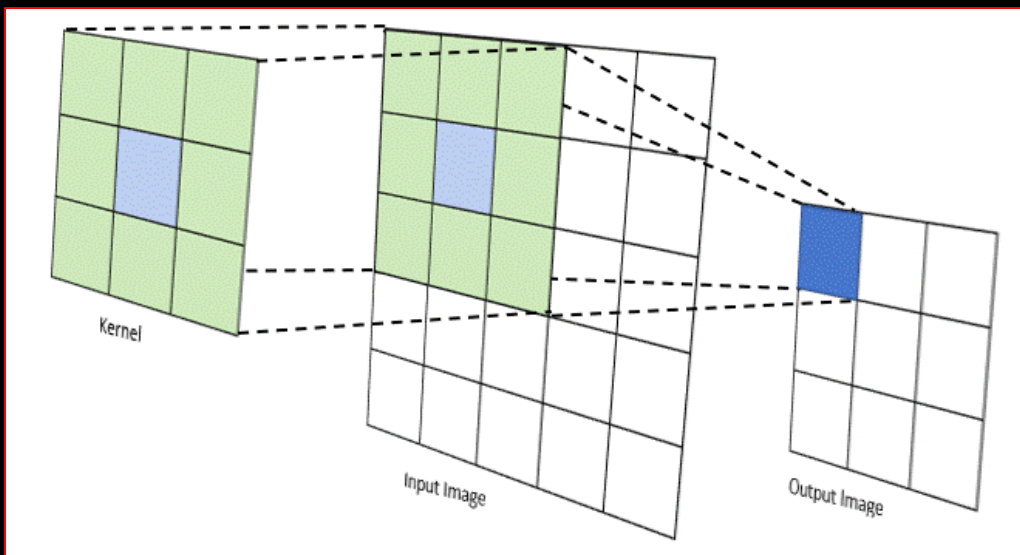
In this cases, Adaptive Median Filtering and Speckle Reduction Bilateral Filtering (SRBF) are used to enhance the quality of image

Figure (a), unwanted artifacts such as a yellow line and the annotation “Ls” display higher intensity than surrounding pixels. If not removed, these artifacts may be falsely emphasized as salient features, leading to inaccurate analysis of the intended object.

CONVOLUTION IN IMAGE FILTERING

Image filtering is a process used to modify or improve digital images through mathematical operations. Filters can smooth out noise, highlight edges or change the way an image looks by applying a set of rules to each pixel and its neighbors. It adjusts digital images by applying mathematical operations to each pixel, allowing for effects such as smoothing, sharpening or edge detection.

In filtering process, we will apply image convolution method. In image convolution, involves a kernel, or matrix that is applied over the input image's pixels to generate an output image. The kernel size and values determine the effect the kernel has on the image. The following illustration defines how image convolution is used to modify image.



NOISE REMOVAL USING ADAPTIVE MEDIAN

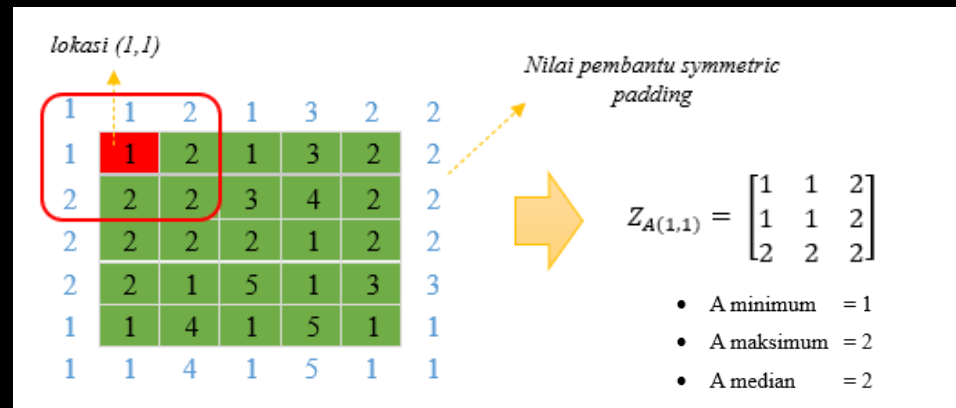
Adaptive Median Filtering is an advanced noise removal technique designed to overcome the limitations of standard median filtering. Instead of using a fixed-size window, the filter adapts the window size based on local noise conditions. Before applying filtering method, the image should be converted in 1 channel (e.g. greyscale). Default value of window of 3x3.

$$A = \begin{bmatrix} 1 & 2 & 1 & 3 & 2 \\ 2 & 2 & 3 & 4 & 2 \\ 2 & 2 & 2 & 1 & 2 \\ 2 & 1 & 5 & 1 & 3 \\ 1 & 4 & 1 & 5 & 1 \end{bmatrix}$$

Pixel value



Original image



$$A_{max} = \begin{bmatrix} 2 & 3 & 4 & 4 & 4 \\ 2 & 3 & 4 & 4 & 4 \\ 2 & 5 & 5 & 5 & 4 \\ 4 & 5 & 5 & 5 & 5 \\ 4 & 5 & 5 & 5 & 5 \end{bmatrix}$$

$$A_{min} = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A_{med} = \begin{bmatrix} 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 1 & 1 & 4 & 1 & 1 \end{bmatrix}$$

Selanjutnya ketiga matriks tersebut dievaluasi apakah sudah memenuhi persyaratan berikut:

- A_{med} lebih besar dari A_{min}
- A_{max} lebih besar dari A_{med}
- A lebih besar dari A_{min}
- A_{max} lebih besar dari A

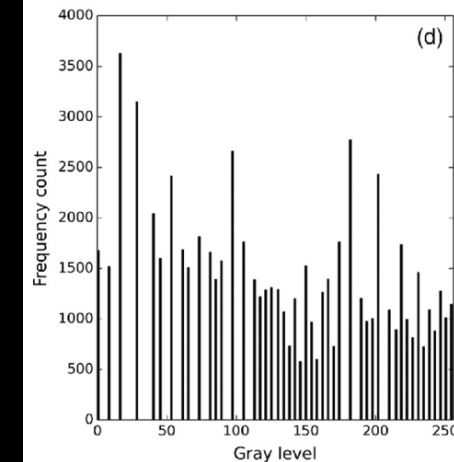
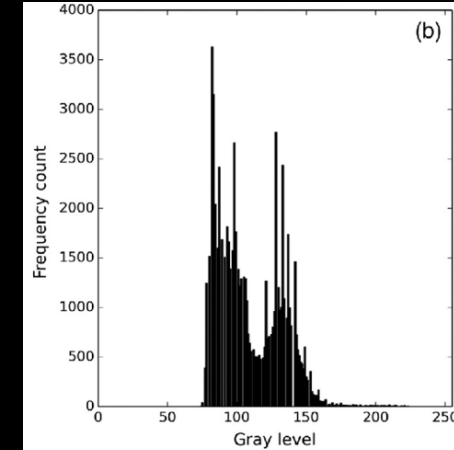
Jika matriks memenuhi syarat maka matriks A_{med} menjadi hasil dari adaptive median. Namun jika matriks belum memenuhi kriteria di atas maka *filtering* dilakukan kembali dengan menggunakan ukuran window 5x5

NOISE REMOVAL

METODE	KEGUNAAN UTAMA	CONTOH KASUS
Mean Filtering	Mereduksi noise acak, namun cenderung mengaburkan detail tepi	Foto pemandangan dengan noise sensor kamera
Median Filtering	Menghilangkan noise salt-and-pepper sambil menjaga ketajaman tepi	Citra dokumen hasil scan dengan bintik hitam/putih
Gaussian Filtering	Mereduksi noise acak dengan perataan berbobot Gaussian	Foto malam dengan noise akibat pencahayaan rendah
Bilateral Filtering	Mengurangi noise tanpa mengorbankan ketajaman tepi	Foto wajah agar tetap halus namun kontur terjaga
Adaptive Median Filtering	Efektif mengatasi noise salt-and-pepper dengan kepadatan tinggi	Citra medis (X-ray/USG) dengan banyak bintik
Wiener Filtering	Mengurangi noise sekaligus mengoreksi blur pada citra	Foto lama yang kabur dan berisik

HISTOGRAM EQUALIZATION

HISTOGRAM EQUALIZATION is an image enhancement technique used to improve the distribution of pixel intensities, thereby increasing the overall contrast of an image.

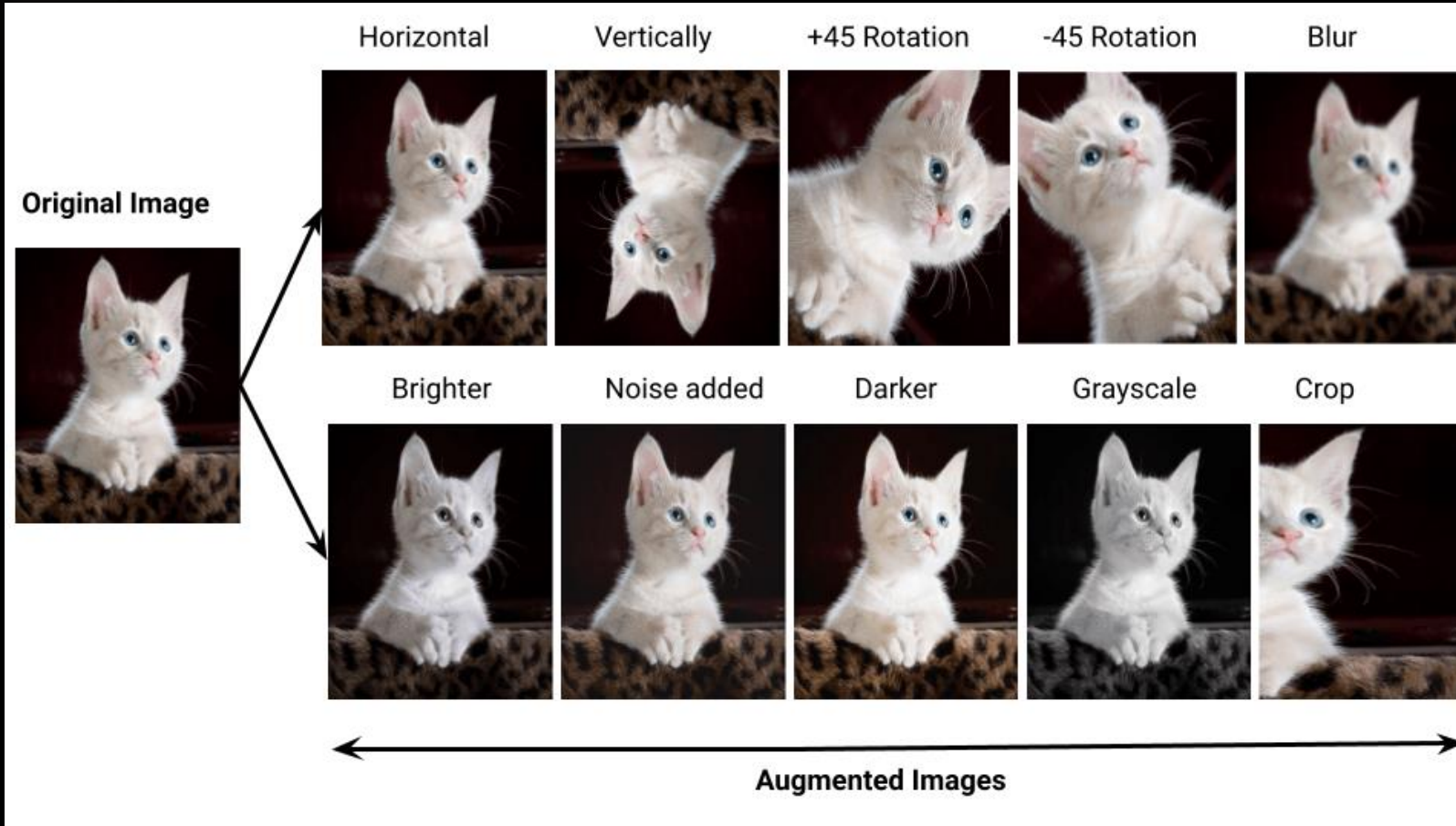


Original image

Equalized image

DATA AUGMENTATION

Data Augmentation is a technique in image processing and machine learning that generates new training samples by applying transformations to existing data, aiming to improve model generalization and reduce overfitting.



Advantages:

- ❑ Expands dataset without additional data collection.
- ❑ Improves robustness and performance of models.
- ❑ Helps models generalize better to unseen data.



IMAGE FILTERING USING PYTHON

IMAGE SHARPENING & BRIGHTENING

```
# Baca gambar
img = cv2.imread(img_path)
# Konversi ke RGB (karena OpenCV default BGR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# --- Sharpening ---
# Kernel sharpening sederhana
kernel_sharpening = np.array([[ -1, -1, -1],
                               [ -1,  9, -1],
                               [ -1, -1, -1]])

sharpened = cv2.filter2D(img_rgb, -1, kernel_sharpening)

# --- Brightening ---
print("Masukkan nilai brightness (misal 50 untuk lebih terang, -50 untuk lebih gelap): ")
brightness_value = int(input())

brightened = cv2.convertScaleAbs(img_rgb, alpha=1, beta=brightness_value)

# --- Tampilkan Hasil ---
plt.figure(figsize=(15,8))

plt.subplot(1,3,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(sharpened)
plt.title("Sharpened Image")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(brightened)
plt.title(f"Brightened ( $\beta$ = {brightness_value})")
plt.axis("off")

plt.show()
```

Original Image



Sharpened Image



Brightened ($\beta=30$)



IMAGE FILTERING

```
# Baca gambar
img = cv2.imread(img_path)
# Konversi ke RGB (karena OpenCV default BGR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# --- Filtering ---

# 1. Mean Filter (Average Blur)
mean_filter = cv2.blur(img_rgb, (5,5)) # kernel 5x5

# 2. Median Filter
median_filter = cv2.medianBlur(img_rgb, 5) # kernel 5

# 3. Gaussian Filter
gaussian_filter = cv2.GaussianBlur(img_rgb, (5,5), 0)

# --- Tampilkan Hasil ---
plt.figure(figsize=(15,10))

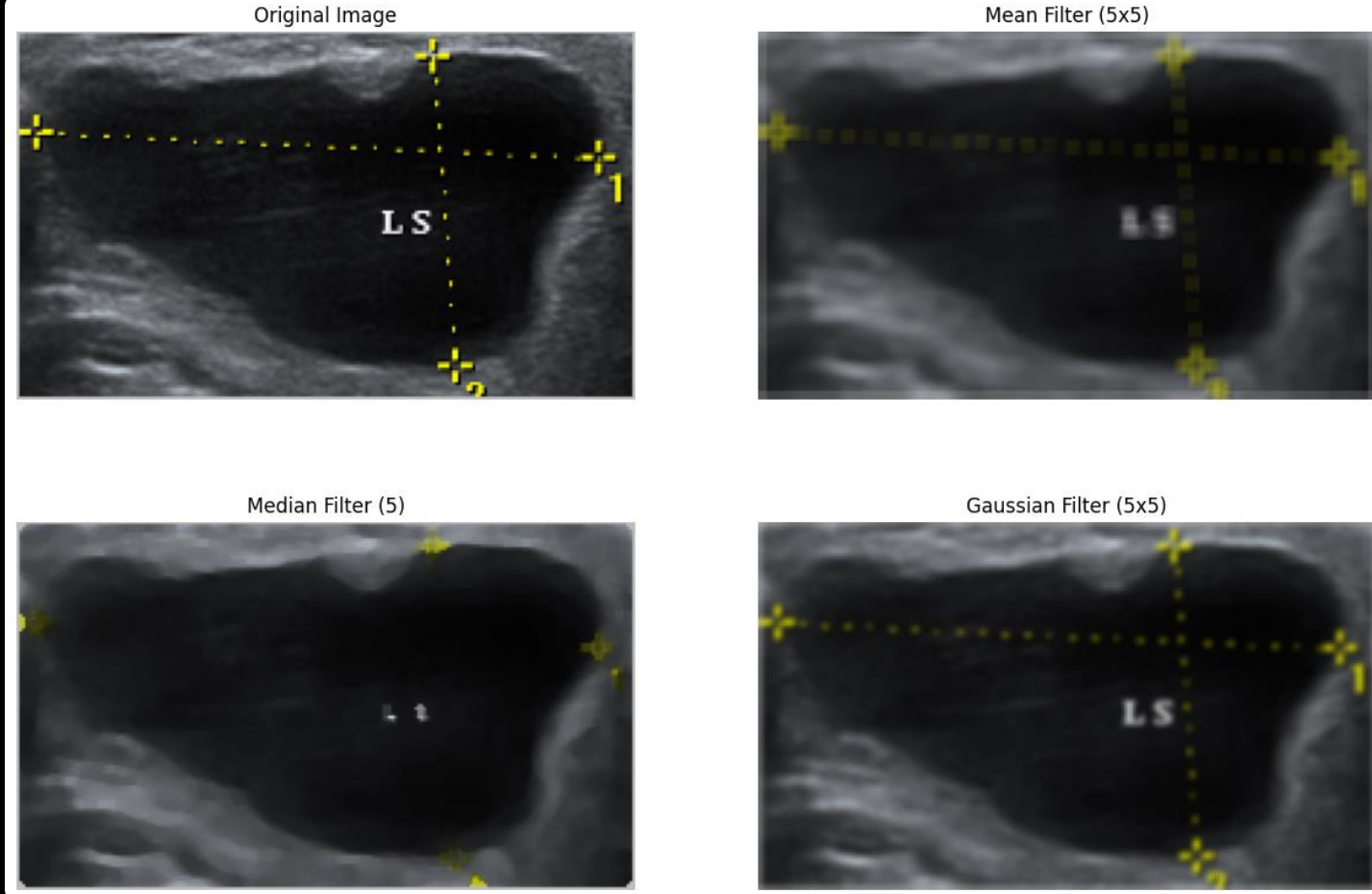
plt.subplot(2,2,1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2,2,2)
plt.imshow(mean_filter)
plt.title("Mean Filter (5x5)")
plt.axis("off")

plt.subplot(2,2,3)
plt.imshow(median_filter)
plt.title("Median Filter (5)")
plt.axis("off")

plt.subplot(2,2,4)
plt.imshow(gaussian_filter)
plt.title("Gaussian Filter (5x5)")
plt.axis("off")

plt.show()
```



This example use kernel size of 5x5. We can adjust the kernel size according to our needs.

IMAGE FILTERING

ADAPTIVE MEDIAN FILTERING

```
# Baca gambar grayscale
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# --- Adaptive Median Filter ---
def adaptive_median_filter(image, max_kernel_size=7):
    padded_img = np.pad(image, max_kernel_size//2, mode='edge')
    output = np.zeros_like(image)

    rows, cols = image.shape
    for i in range(rows):
        for j in range(cols):
            k = 3 # mulai dari kernel 3x3
            pixel_processed = False

            while k <= max_kernel_size and not pixel_processed:
                window = padded_img[i:i+k, j:j+k].flatten()
                Zmin = np.min(window)
                Zmax = np.max(window)
                Zmed = np.median(window)
                Zxy = padded_img[i + k//2, j + k//2]

                A1 = Zmed - Zmin
                A2 = Zmed - Zmax

                if A1 > 0 and A2 < 0:
                    B1 = Zxy - Zmin
                    B2 = Zxy - Zmax
                    if B1 > 0 and B2 < 0:
                        output[i, j] = Zxy
                    else:
                        output[i, j] = Zmed
                        pixel_processed = True
                else:
                    k += 2 # perbesar kernel ukuran ganjil berikutnya

            if not pixel_processed:
                output[i, j] = Zmed # fallback
    return output
```

```
# Terapkan filter
filtered_img = adaptive_median_filter(img, max_kernel_size=7)

# --- Tampilkan Hasil ---
plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(filtered_img, cmap='gray')
plt.title("Adaptive Median Filtered")
plt.axis("off")

plt.show()
```

This method is commonly used for noise removal purposes. The process typically operates on grayscale images, since reducing the image to a single intensity channel simplifies computation and avoids the need to process each color channel separately.

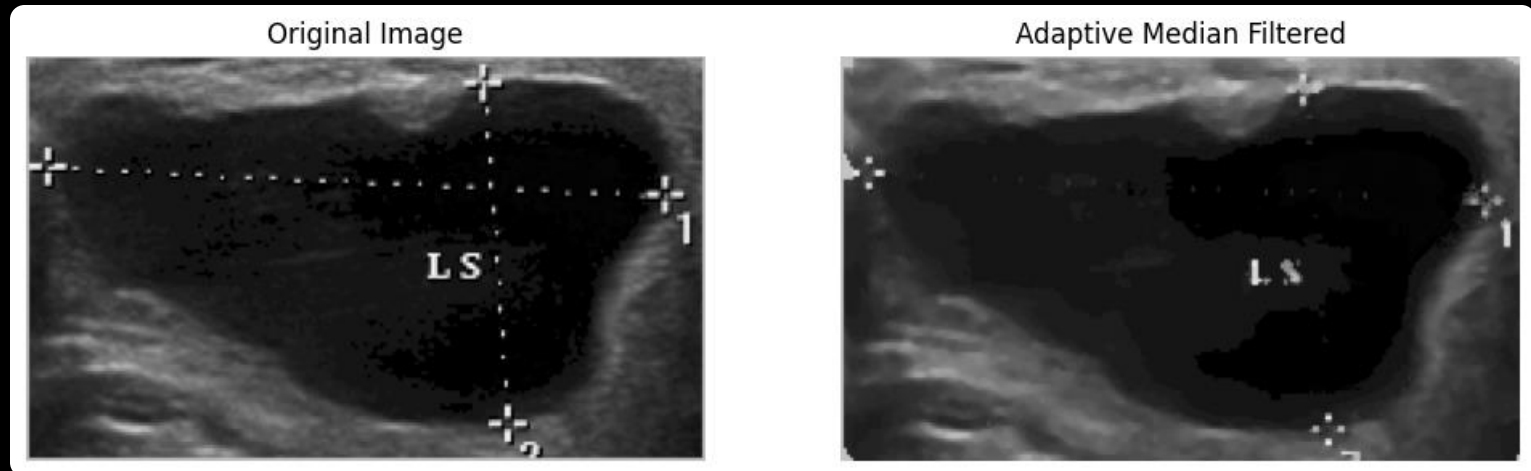


IMAGE FILTERING

BILATERAL FILTERING

```
# Step 3: Read image
image = cv2.imread(filename)

# Convert from BGR (OpenCV default) to RGB (for visualization)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Step 4: Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 5: Apply Bilateral Filtering
# Parameters: (src, d, sigmaColor, sigmaSpace)
# d = diameter of pixel neighborhood
# sigmaColor = filter sigma in the color space
# sigmaSpace = filter sigma in the coordinate space
bilateral = cv2.bilateralFilter(gray, d=9, sigmaColor=75, sigmaSpace=75)

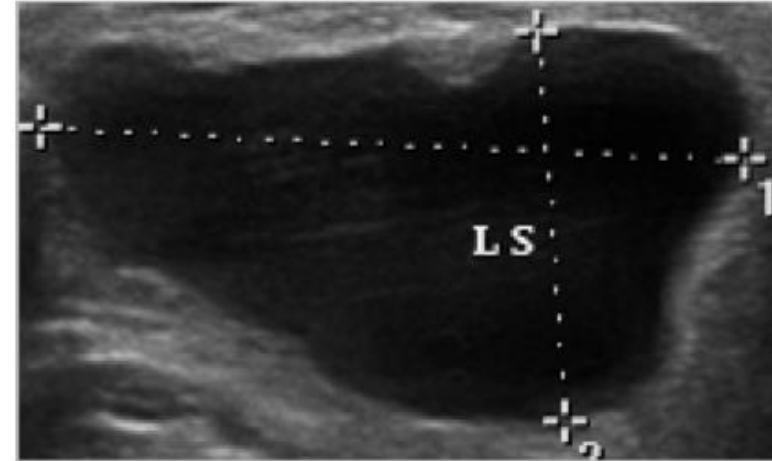
# Step 6: Show results
plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(bilateral, cmap='gray')
plt.title("Bilateral Filtered Image")
plt.axis("off")

plt.show()
```

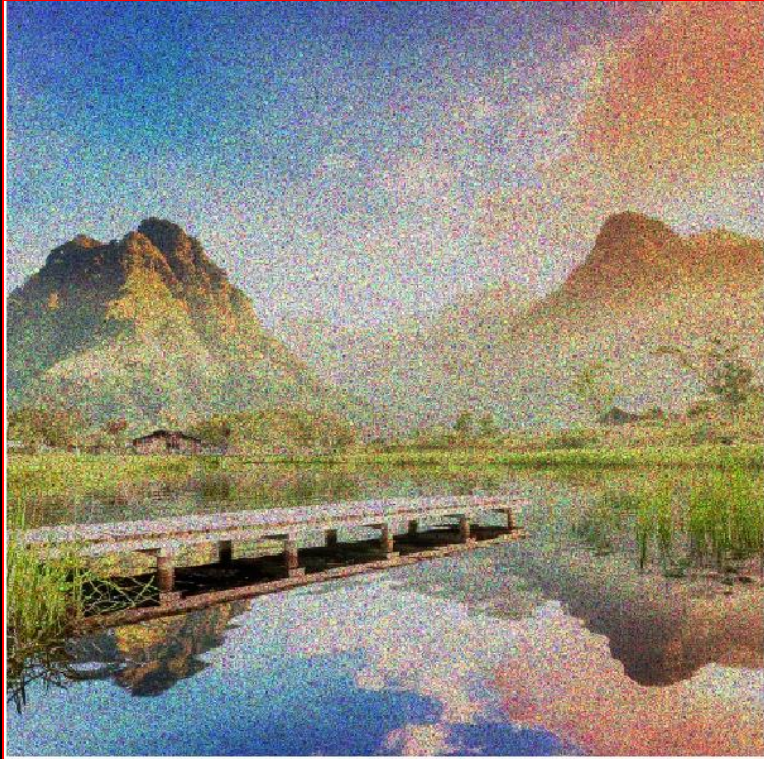
Original Grayscale Image



Bilateral Filtered Image



TUGAS



Lakukan filtering pada ketiga gambar di atas:

1. Hapus noise speckle pada gambar 1 dan 3
2. Hapus noise anotasi pada gambar 2
3. Terapkan image enhancement (image brightening, sharpening, blurring, dll) untuk mempertajam detail gambar
4. Implementasikan metode image augmentation